



Instituto Tecnológico de Costa Rica

IC4700 Lenguajes de Programación

Operación Atlas - Prolog

Proyecto 03

Emilio Funes R.

Ginger Rodriguez G.

Jareck Levell C.

Grupo 60

Profesor: Allan Rodriguez Davila

I Semestre

2026

1. Manual de usuario

1.1 Requisitos para ejecutar el proyecto

- Tener instalado SWI-Prolog para ejecutar el motor lógico del juego.
- Tener instalado Node.js y npm para ejecutar el backend y el frontend web.
- Usar un navegador web moderno, como Microsoft Edge, Google Chrome o Firefox.
- Mantener la estructura del proyecto con las carpetas Programa/backend, Programa/frontend y Programa/prolog.

1.2 Ejecución del backend

Desde una terminal ubicada en la carpeta Programa/backend, ejecutar:

```
npm install
```

```
npm run start
```

El backend funciona como puente entre la interfaz web y SWI-Prolog. Por defecto, el servidor queda disponible en el puerto 3002.

1.3 Ejecución del frontend

Desde una segunda terminal ubicada en la carpeta Programa/frontend, ejecutar:

```
npm install
```

```
npm run dev
```

Luego se abre en el navegador la dirección local indicada por Vite, normalmente <http://localhost:5173/>.

1.4 Uso general del juego

El jugador inicia en el menú principal de Operación Atlas. Al seleccionar Iniciar juego, se abre una terminal de ingeniería conectada a la Estación Aurora. La interacción se realiza escribiendo comandos en la consola. Cada comando se envía al backend, y este lo traslada al motor lógico desarrollado en Prolog.

Comandos principales:

estado_juego.

acciones_disponibles.

tomar(Artefacto).

usar(Artefacto).

mover(Modulo).

reparar(Sistema).

rescatar(Tripulante).

ruta(Inicio, Fin, Camino).

como_gano.

verifica_gane.

2. Pruebas de funcionalidad

La siguiente tabla resume las pruebas mínimas sugeridas. En la versión final del documento se pueden insertar capturas de pantalla debajo de cada prueba o al final de esta sección.

Prueba	Comando o acción	Resultado esperado
Inicio del juego	Iniciar juego	Se muestra la terminal con el lore inicial de la misión.
Consulta de estado	estado_juego.	Se muestra ubicación, inventario, sistemas, tripulantes y módulos visitados.
Consulta de acciones	acciones_disponibles.	Se muestran las acciones posibles según el estado actual.
Bloqueo lógico	mover(modulo_energia).	La acción falla si no se ha usado el traje espacial.
Toma de artefacto	tomar(tarjeta_seguridad).	La tarjeta se agrega al inventario si está en el módulo actual.
Reparación	reparar(energia).	El sistema cambia de fallo a restaurado si se cumplen las condiciones.
Rescate	rescatar(elena).	El tripulante cambia de atrapado a rescatado si se cumplen los sistemas requeridos.
Victoria	verifica_gane.	Si todos los

		objetivos cumplieron, muestra condición victoria.	se se la de
--	--	---	----------------------

3. Descripción del problema

Operación Atlas es un juego de aventura lógica ambientado en la Estación Aurora. Después de una tormenta solar, varios módulos de la estación quedan comprometidos y algunos tripulantes permanecen atrapados. El jugador asume el rol de ingeniero principal y debe explorar la estación, recolectar artefactos, reparar sistemas críticos, rescatar tripulantes y cumplir las condiciones de victoria.

El problema se resuelve con el paradigma lógico porque el mundo del juego se representa mediante hechos y reglas. Los hechos describen módulos, enlaces, artefactos, sistemas, tripulantes y restricciones. Las reglas permiten razonar si una acción es válida, si un módulo puede visitarse, si un sistema puede repararse o si la misión ya fue completada.

4. Diseño del programa

4.1 Organización general

El proyecto se divide en tres capas principales:

- Prolog: contiene la base de conocimiento, los hechos y las reglas del juego.
- Backend Node.js: actúa como puente entre la web y Prolog. No resuelve la lógica del juego.
- Frontend React/Vite: presenta la interfaz gráfica, recibe comandos del usuario y muestra las respuestas.

4.2 Organización de Prolog

```
prolog/
  main.pl
  hechos.pl
  reglas.pl
  adaptador_web.pl
  reglas/
    listas.pl
    estado.pl
    movimiento.pl
    artefactos.pl
    sistemas.pl
    tripulantes.pl
    rutas.pl
```

objetivos.pl
control.pl

La decisión principal fue separar los hechos del mundo y las reglas del motor lógico. Esto permite que el profesor pueda cambiar los hechos del juego mientras las reglas se mantienen funcionando, siempre que se respete la estructura de predicados establecida.

4.3 Decisiones de diseño

- Los enlaces se interpretan como bidireccionales mediante el predicado conectado/2.
- El inventario se guarda como una lista dentro de artefactosLogrados/1.
- El historial de módulos se actualiza cada vez que se ejecuta mover/1 exitosamente.
- Las restricciones se dividen en artefactos requeridos, estados de sistemas y pasos previos.
- La interfaz web no conoce la lógica del juego; solamente envía comandos y muestra respuestas.

5. Análisis de resultados

5.1 Objetivos alcanzados

- Se implementó un juego de aventura espacial con lógica principal en Prolog.
- El jugador puede moverse entre módulos conectados y restringidos por condiciones lógicas.
- El jugador puede tomar y usar artefactos.
- El jugador puede reparar sistemas críticos si posee los artefactos requeridos y está en el módulo correcto.
- El jugador puede rescatar tripulantes si se cumplen las condiciones necesarias.
- Se implementó búsqueda de rutas mediante recursión y backtracking.
- Se implementó una interfaz web que se comunica con Prolog mediante un backend puente.
- Se implementó verificación de victoria con resumen final de ruta, artefactos, sistemas y tripulación.

5.2 Objetivos no alcanzados

No se implementó la funcionalidad opcional de guardar y reproducir una repetición en disco duro. Esta característica no formaba parte de los requisitos obligatorios y se priorizó completar correctamente la funcionalidad principal del juego, la conexión web-Prolog y la documentación.

6. Análisis de predicados complejos

6.1 ruta/3

El predicado ruta/3 encuentra un camino lógico entre dos módulos. Utiliza un predicado auxiliar que recibe el módulo actual, el destino, una lista de visitados y el camino resultante. La lista de visitados evita ciclos, y el backtracking permite que Prolog pruebe rutas alternativas cuando una conexión no lleva al destino.

6.2 puedo_ir/1 y mover/1

puedo_ir/1 determina si el jugador puede desplazarse al destino indicado. Para lograrlo, verifica que exista conexión, que se cumplan los artefactos requeridos, que los sistemas necesarios estén restaurados y que los pasos previos se hayan visitado. mover/1 utiliza puedo_ir/1 antes de actualizar la ubicación del jugador y el historial de módulos. Así se evita que el jugador avance si no cumple las restricciones lógicas.

6.3 como_gano/0

como_gano/0 funciona como una guía lógica del juego. No juega automáticamente por el usuario, sino que analiza qué sistemas y tripulantes siguen pendientes. También muestra artefactos requeridos, rutas posibles y restricciones. Este predicado es importante porque integra información de varios hechos y reglas para orientar al jugador sin romper la lógica de la aventura textual.

7. Conclusión

El proyecto Operación Atlas cumple con el objetivo de aplicar el paradigma lógico en un juego interactivo. La solución representa el mundo mediante hechos y reglas, utiliza listas, recursión, unificación y backtracking, y separa correctamente la lógica en Prolog de la interfaz web. La implementación permite demostrar movimiento, restricciones, artefactos, reparación, rescate, rutas y condición de victoria.